



LocaCart

Support Local Vendors. Shop Smart.

Order from nearby shops, enjoy fast delivery, and use Pay Later.

[Show Vendors](#)[Show Products](#)



Shop from Nearby Vendors

[Search](#)

Shops Near You



Gupta vegetables and fruits

 Lanka

[Explore Shop](#)[Activate Pay Later](#)

Mahesh grocery shop

 Lanka

[Explore Shop](#)

Credit Amount: ₹5000



Santosh general store

 Lanka

[Explore Shop](#)

Credit Amount: ₹4804



tester shop

 Lanka

[Explore Shop](#)[Activate Pay Later](#)

Store Overview

10

LIVE PRODUCTS

5

ACTIVE ORDERS

₹83.0

PAYMENT DUE

Common Tasks

+

Add Item



Products



Orders



Pay Later

Products Preview

[Manage →](#)



Strawberry

₹320

1 kg



Radish

₹40

1 kg



Bitter Gourd

₹68

1 kg



Green Chilli

₹160

1 kg



Broccoli

₹39

1 piece



Onion

₹27

1 kg

TOTAL MARKET DUE

₹394

ADVANCE BALANCE

₹245

ACTIVE CUSTOMERS

4



Santosh general store

 Santosh Singh

 Lanka



Manage Requests

Approve or reject customer requests for credit accounts.



Manage Customers

View credit profiles, update limits, and record payments.



Transaction History

Detailed log of all credit purchases and cash repayments.

Order History

✓ Accepted! Share PIN on delivery.

 **SANTOSH GENERAL STORE**

ACCEPTED

#ORD-E6E23E9F

18 Apr, 2026

 **CASH ON DELIVERY**



Amul Masti Dahi (Cup)

1 unit • ₹92



Amul Taaza Milky Milk

1 unit • ₹58



Amul Gold Pasteurised Full Cream Milk

1 unit • ₹35



Bran-O Plus Brown Bread

1 unit • ₹35

TOTAL AMOUNT

₹220

DELIVERY PIN

9773



A PROJECT REPORT
ON
" LOCAL VENDOR HELPING SYSTEM "



Submitted in partial fulfillment of the requirements for the three years

BACHELOR OF
COMPUTER APPLICATIONS

(Affiliated to Mahatma Gandhi Kashi Vidyapith Varanasi, Uttar Pradesh)

SESSION-2025-26

Under Supervision of:

MS. SRIPRIYA ROY CHOWDHURI
(Assistant Professor)

Submitted By:

Rajnish Dwivedi(BCA2326109)
Abhishek Kumar(BCA2326009)
Piyush Gupta(BCA2326095)
Sakshi Kumari(BCA2326123)



TABLE OF CONTENT

S. NO	CONTENTS	PAGE NO.
1.	Introduction	1-4
	Present Scenario of the Project	1
	Problems in the Existing System	2
	Solution of the problem	2-3
	Objective of the project	3-4
2.	Feasibility Study	5-6
	Technical Feasibility	5
	Operational Feasibility	5-6
3.	Software Requirement Specifications	7-9
	Hardware Requirement	7
	Software Requirement	7-8
	Platform / OS Support	8-9
4.	System Design	10-15
	Data Flow Diagram	10-14
	E-R Diagram	15
5.	Code Sample	16-25
6.	Software Screen	26-30
7.	Conclusion	31
8.	Future Scope	32
9.	Bibliography	33

INTRODUCTION

1.1 Present Scenario of the Project

The present scenario of the Local Vendor Helping System highlights a rapidly evolving retail environment where digital transformation has become essential for survival and growth. With the increasing penetration of smartphones, affordable internet access, and digital payment systems across India, customers are now more inclined towards online purchasing, even for their daily household needs. Platforms like Blinkit, Zepto, and Swiggy Instamart have set new standards for convenience, speed, and user experience, making it challenging for traditional vendors to compete in this changing ecosystem.

Currently, most local vendors continue to operate using traditional methods such as physical stores, manual billing, and limited customer outreach. They rely heavily on walk-in customers and word-of-mouth marketing, which restricts their growth potential in a highly competitive market. While some vendors have started using basic digital tools like WhatsApp for order taking, they still lack an integrated system for inventory management, customer tracking, order processing, and digital credit handling. This gap in technology adoption has created an imbalance where large platforms dominate customer attention, leaving small vendors struggling to maintain their market presence.

At the same time, customers are facing certain limitations with large online platforms, such as higher prices, limited personalization, and reduced trust compared to their local neighbourhood shops. Many customers still prefer local vendors for their reliability, flexible credit options, and personal relationships, but they expect the same level of convenience as digital platforms. This creates a strong opportunity for a system that combines the trust of local vendors with the efficiency of modern technology.

The Local Vendor Helping System fits perfectly into this present scenario by addressing both vendor and customer needs. It provides a structured digital solution where vendors can easily register, list products, manage orders, and maintain customer relationships digitally without requiring advanced technical knowledge. At the same time, customers can browse nearby vendors, place orders, and avail features like “Pay Later” in a secure and organized way. Therefore, the system aligns with current market demands by enabling local vendors to adapt to digital trends while preserving their traditional strengths.

1.2 Problems in the Existing System

The existing retail ecosystem faces several critical problems due to the dominance of large online delivery platforms and the lack of digital infrastructure among local vendors.

Firstly, local shopkeepers are losing customers as people increasingly prefer online apps that offer doorstep delivery, quick service, and digital payments. Traditional vendors, who rely solely on physical visits, are unable to match this level of convenience.

Secondly, local vendors lack technological support. Most operate without digital product catalogs, automated order management, or online visibility. This makes it difficult for them to attract digitally active customers, especially younger consumers.

Thirdly, vendors who attempt to join large delivery platforms often face high commission charges, strict policies, delayed payments, and loss of control over customer relationships. These platforms prioritize profits, leaving small vendors with reduced margins.

Additionally, the traditional manual credit system is error-prone and lacks transparency. Handwritten records can lead to disputes, forgotten dues, and financial mismanagement. Customers also have no digital way to track their outstanding balances.

If these issues remain unaddressed, local vendors risk reduced footfall, declining revenue, and eventual closure, leading to loss of employment and weakening of local economies

1.3 Solution to the problem

The **Local Vendor Helping System** provides a comprehensive and practical solution to the challenges faced by local shopkeepers by introducing a **hyper-local digital marketplace** tailored specifically for neighbourhood commerce.

Instead of competing with large platforms, the system brings local vendors online within their own locality, allowing them to serve customers they already know and trust. Customers can browse nearby vendors, view product catalogs, place orders, and track deliveries digitally, while vendors manage everything through a simple dashboard.

A key innovation of the system is the Pay Later feature, which digitizes the traditional credit system used in Indian markets. Customers can request Pay Later access from vendors, and vendors can approve

or reject requests based on trust. Vendors can set credit limits, track outstanding dues, and record offline or online payments digitally, ensuring full transparency and financial control.

The system also supports easy returns and replacements, which are faster and more flexible due to the hyper-local nature of vendors. Delivery charges remain minimal or zero since vendors operate within nearby areas, making the service affordable for customers.

An Admin module ensures system integrity by verifying vendors, managing user records, and generating reports related to orders, sales, and Pay Later transactions. This maintains trust, authenticity, and smooth system operation.

In summary, the Local Vendor Helping System successfully addresses existing problems by combining technology with trust, empowering vendors, preserving neighbourhood relationships, and providing customers with convenience, affordability, and transparency all without the burden of heavy commissions or external dependencies.

1.4 Objective of the project

The objectives of the Local Vendor Helping System are clearly defined to solve real-world challenges faced by local vendors while enhancing the overall shopping experience for customers. Unlike traditional problem descriptions, this section focuses on what the system aims to achieve rather than what is happening currently.

The first and foremost objective of the project is to provide a dedicated digital platform for local vendors where they can register their shops and create an online presence without depending on third-party platforms. This ensures that vendors can independently manage their business, display their products, and directly interact with customers without paying high commissions or losing control over pricing and customer relationships.

Another key objective is to enable hyper-local connectivity between customers and nearby vendors. The system is designed in such a way that customers can easily find shops within their locality, view available products, and place orders instantly. This reduces delivery time, lowers costs, and strengthens neighbourhood-level commerce, which is not the primary focus of large-scale platforms.

The project also aims to digitize the traditional credit system through the “Pay Later” feature. Instead of maintaining handwritten records, vendors can manage customer credit digitally by setting limits,

tracking dues, and recording payments. This objective is important because it brings transparency, reduces errors, and preserves trust between vendors and customers in a modern format.

Another important objective is to simplify business operations for vendors. The system provides tools for order management, inventory tracking, and customer handling in a structured way. This reduces manual workload, improves accuracy, and allows vendors to focus more on business growth rather than record-keeping.

Additionally, the project aims to enhance customer convenience by offering features such as online ordering, easy product browsing, quick delivery, and flexible payment options. Customers get the benefits of both worlds the trust of local vendors and the convenience of digital platforms.

Finally, the system is designed with the objective of supporting and strengthening the local economy. By empowering small vendors with digital tools, the project helps them compete in the modern market, prevents business decline, and promotes sustainable economic growth at the community level.

FEASIBILITY STUDY

2.1 Technical Feasibility

Technical feasibility evaluates whether the proposed system can be developed and implemented using the available technology, tools, infrastructure, and technical skills.

The Local Vendor Helping System is technically feasible because it is designed using widely available, reliable, and cost-effective technologies that are well suited for web-based application development. The system is developed using HTML, CSS, and JavaScript for the frontend, Python Django for the backend, and SQLite as the database, all of which are proven and stable technologies.

Django provides strong support for rapid development, secure authentication, database integration, and scalability. Features such as user authentication (Customer, Vendor, Admin), order processing, Pay Later credit management, and report generation can be efficiently implemented using Django's built-in modules and ORM (Object Relational Mapping). SQLite is sufficient for academic and small-scale deployment, while the system architecture allows easy migration to MySQL or PostgreSQL in the future if required.

The hardware requirements are minimal and easily available, such as an Intel i5 processor, 6 GB RAM, and basic internet connectivity, making the system deployable on standard institutional or personal machines. No specialized hardware or expensive infrastructure is required.

Additionally, the system follows the Prototype Model, which ensures early testing of modules such as vendor dashboards, order flow, and Pay Later functionality. This reduces development risk and allows technical issues to be identified and resolved at an early stage.

Therefore, from a technical perspective, the project is feasible, implementable within academic constraints, and scalable for future enhancements such as mobile app integration or AI-based analytics.

2.2 Operational Feasibility

Operational feasibility examines whether the system can be successfully used, accepted, and operated by its intended users in a real-world environment.

The Local Vendor Helping System is operationally feasible because it is designed with simplicity, usability, and real-world practices of local vendors and customers in mind. Most local vendors already

understand basic smartphone usage and daily business operations; the system digitizes these familiar processes rather than introducing complex workflows.

For customers, the system offers easy registration, simple product browsing, order placement, order tracking, and a familiar Pay Later concept in digital form. Since customers already trust their local vendors, adoption resistance is minimal. Features such as easy returns and minimal delivery charges further improve user acceptance.

For vendors, the system replaces manual registers with a digital dashboard that manages products, orders, payments, and credit accounts. Vendors maintain full control over pricing, customer approvals, and credit limits, ensuring they do not feel dependent on external platforms or administrators. The Pay Later feature aligns with traditional Indian market practices, making it easy for vendors to adopt without operational disruption. The Admin module ensures smooth operation by verifying vendors, monitoring system activities, and generating reports. This helps maintain trust, data accuracy, and system discipline.

Training requirements are minimal, as the user interface is intuitive and role-specific. The system can be operated without hiring additional staff, reducing operational costs. Furthermore, since the platform eliminates heavy commissions and middlemen, vendors experience direct operational benefits.

Thus, the system is operationally viable, socially acceptable, easy to use, and capable of functioning effectively in real-life neighbourhood retail environments.

SOFTWARE REQUIREMENT SPECIFICATIONS

4.1 Hardware Requirement

The following hardware requirements are necessary for the development, testing, and deployment of the Local Vendor Helping System. These requirements are minimal and easily available, ensuring cost-effective implementation.

Server / Development System

- **Processor:** Intel i5 or higher
- **RAM:** Minimum 6 GB
- **Hard Disk:** Minimum 256 GB
- **Internet Connection:** Stable broadband connection for development and testing

Client System (Customer / Vendor / Admin)

- **Processor:** Any standard processor (Intel i3 or equivalent)
- **RAM:** Minimum 4 GB
- **Device:** Desktop, Laptop, or Smartphone
- **Internet Connectivity:** Required for accessing the web application

The system does not require any specialized or high-end hardware, making it suitable for academic environments as well as small-scale real-world deployment.

4.2 Software Requirement

The Local Vendor Helping System is developed using open-source and widely supported software technologies to ensure flexibility, security, and scalability.

Frontend Technologies

- **HTML5:** Used to structure web pages and user interfaces
- **CSS3:** Used for styling, layout design, and responsive user interface

- **JavaScript:** Used for client-side validation and interactive features

Backend Technologies

- **Python:** Core programming language used for backend development
- **Django Framework:**
 - Handles server-side logic
 - Provides authentication, session management, and security
 - Supports rapid development and scalability

Database

- **SQLite:**
 - Used to store customer, vendor, product, order, and Pay Later data
 - Lightweight and suitable for academic and small-scale deployment
 - Can be upgraded to MySQL or PostgreSQL in future

Development Tools

- **Visual Studio Code:** Code editor for development
- **Web Browser:** Google Chrome for testing
- **Operating System:** Windows 10 or above

4.3 Platform / OS Support

The Local Vendor Helping System is designed as a web-based application that ensures compatibility across multiple platforms and operating systems, making it easily accessible for vendors, customers, and administrators. The system is developed using standard web technologies such as HTML, CSS, JavaScript for the frontend, and frameworks like Django for the backend, which allows it to run smoothly on any device that supports a modern web browser.

The application supports major operating systems including Microsoft Windows, Linux, macOS, Android, and iOS. This cross-platform compatibility ensures that users can access the system from

desktops, laptops, tablets, and smartphones without any restrictions. Vendors can manage their shop using a computer or mobile device, while customers can conveniently browse and place orders using their smartphones.

In terms of browser support, the system is compatible with widely used web browsers such as Google Chrome, Mozilla Firefox, Microsoft Edge, and Safari. The use of responsive web design ensures that the user interface automatically adjusts to different screen sizes and resolutions, providing a consistent and user-friendly experience across all devices.

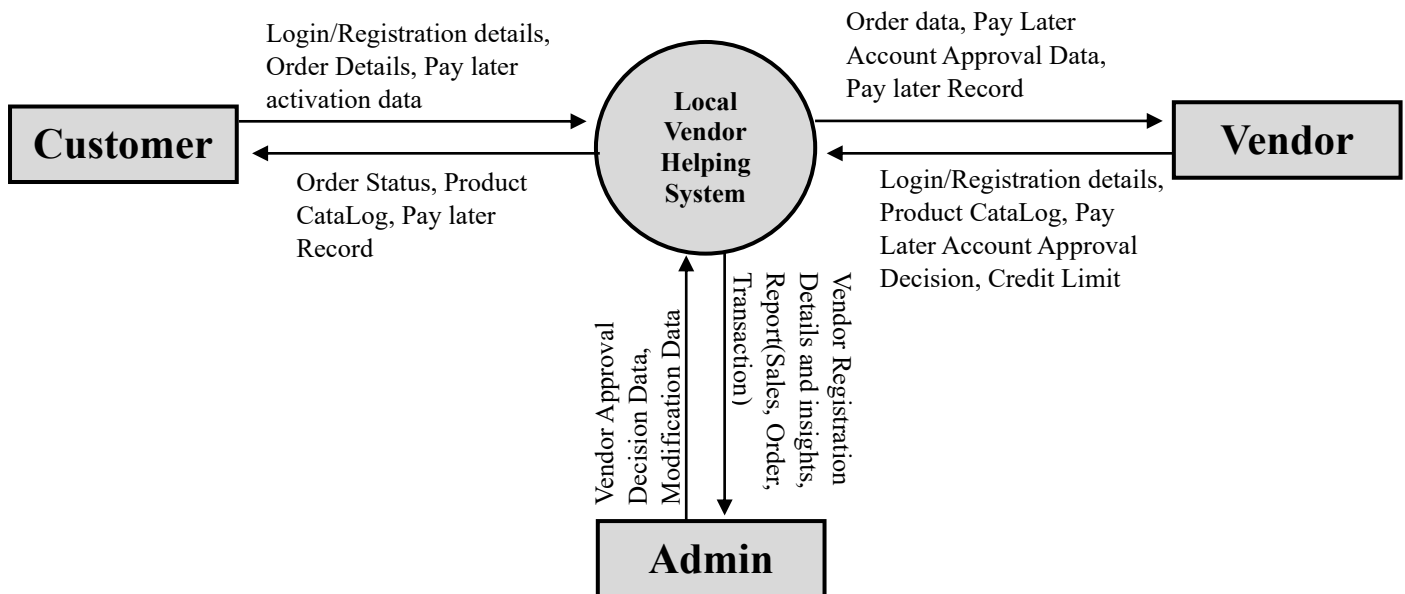
Additionally, the system requires minimal hardware resources, making it suitable even for low to mid-range devices commonly used by local vendors. Since it is a cloud-based or server-hosted application, users do not need to install any heavy software; they only require an internet connection and a web browser to access the system.

Overall, the platform independence and wide OS support of the Local Vendor Helping System make it highly scalable, accessible, and practical for real-world implementation, especially in diverse environments where users may have different types of devices and technical capabilities.

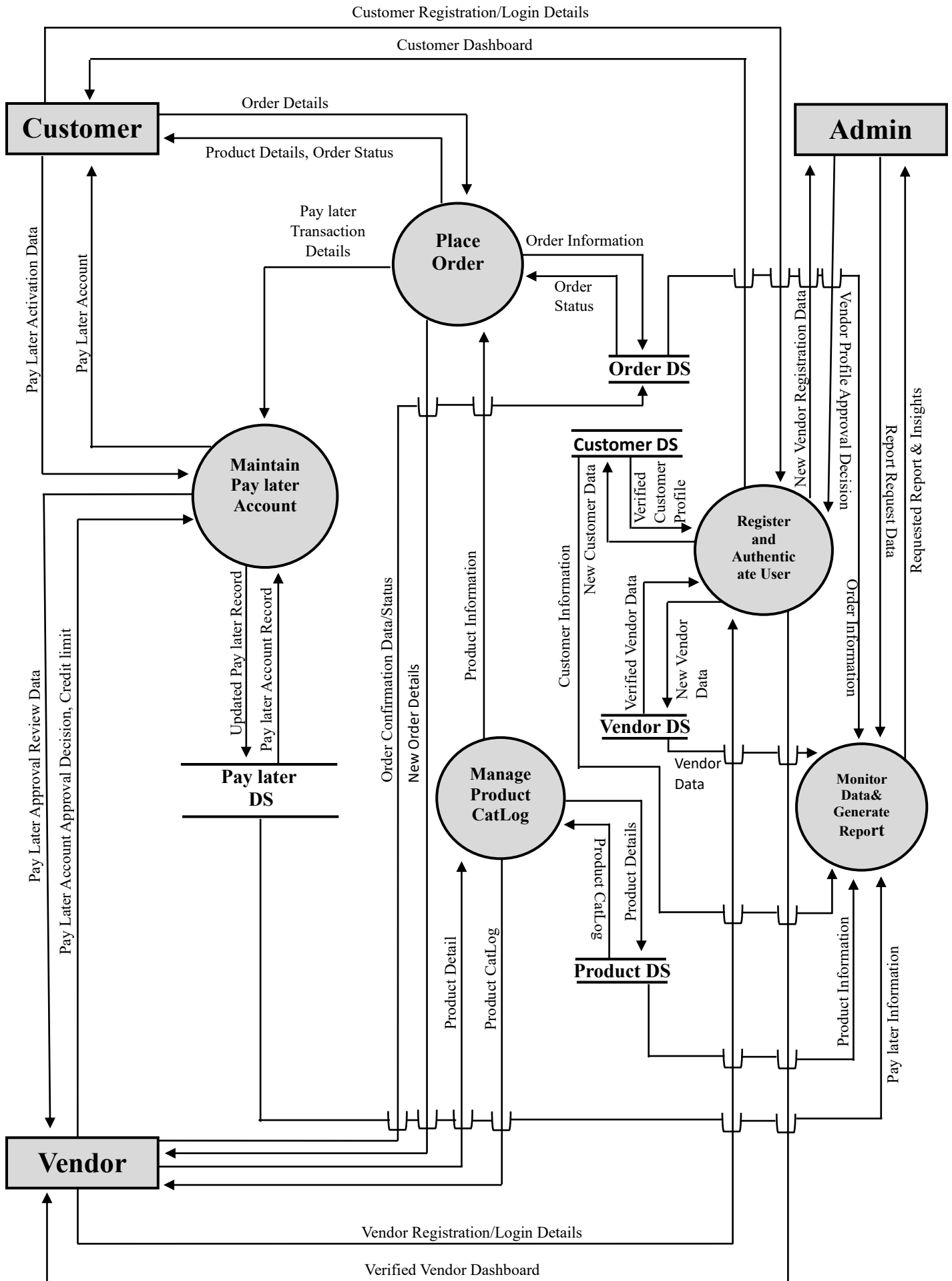
SYSTEM DESIGN

5.1 Data Flow Diagram (DFD)

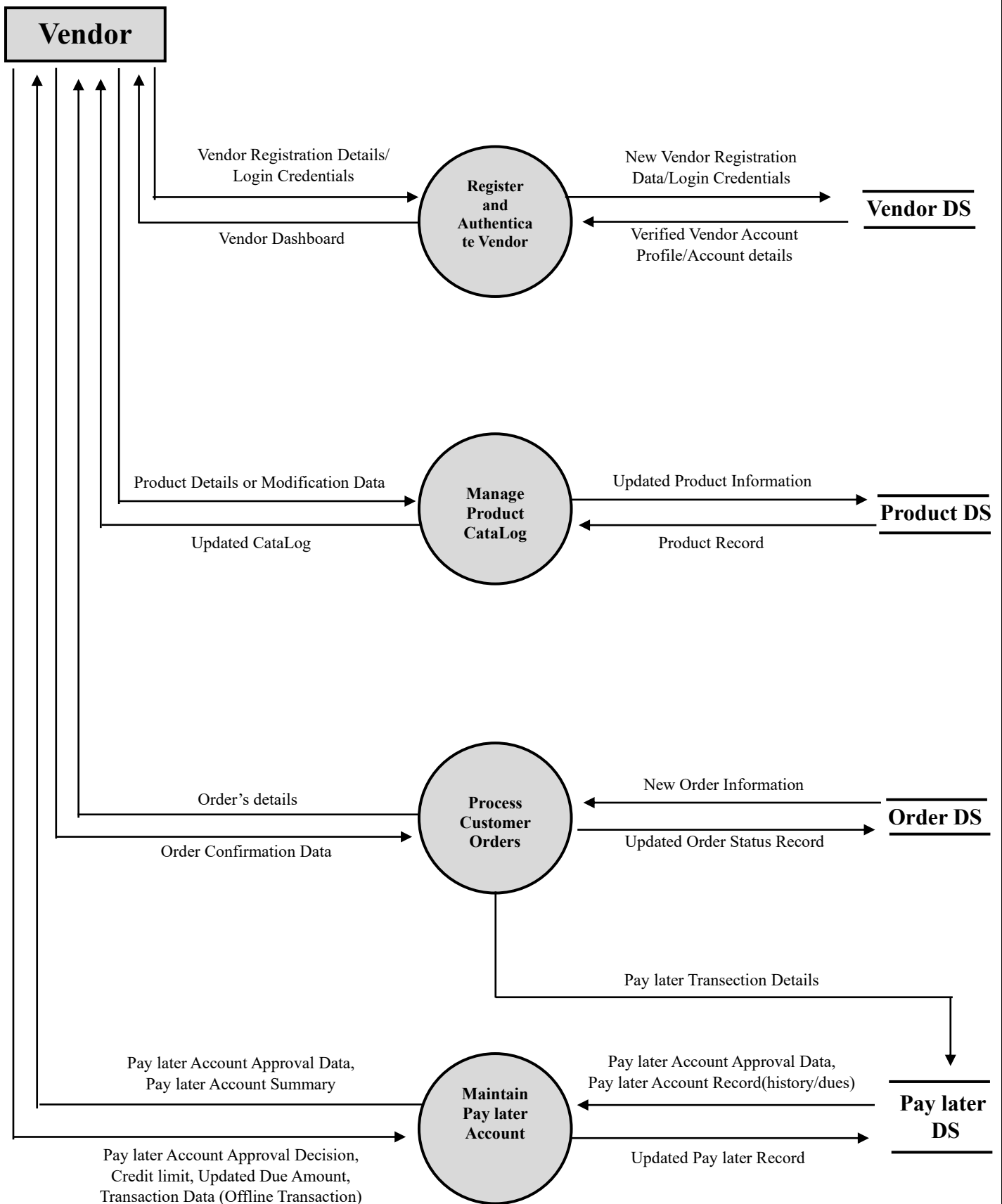
LEVEL '0' DFD



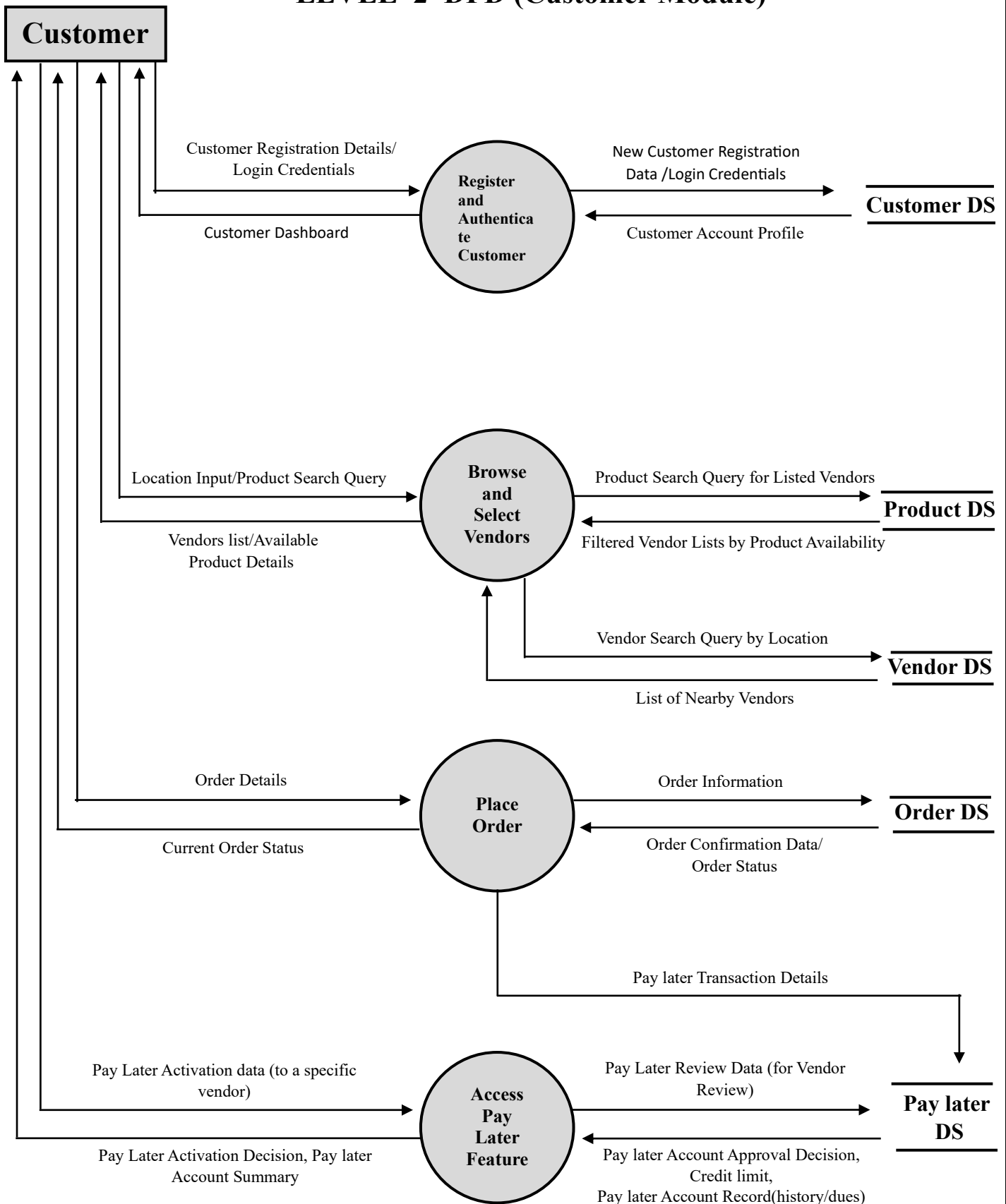
LEVEL '1' DFD



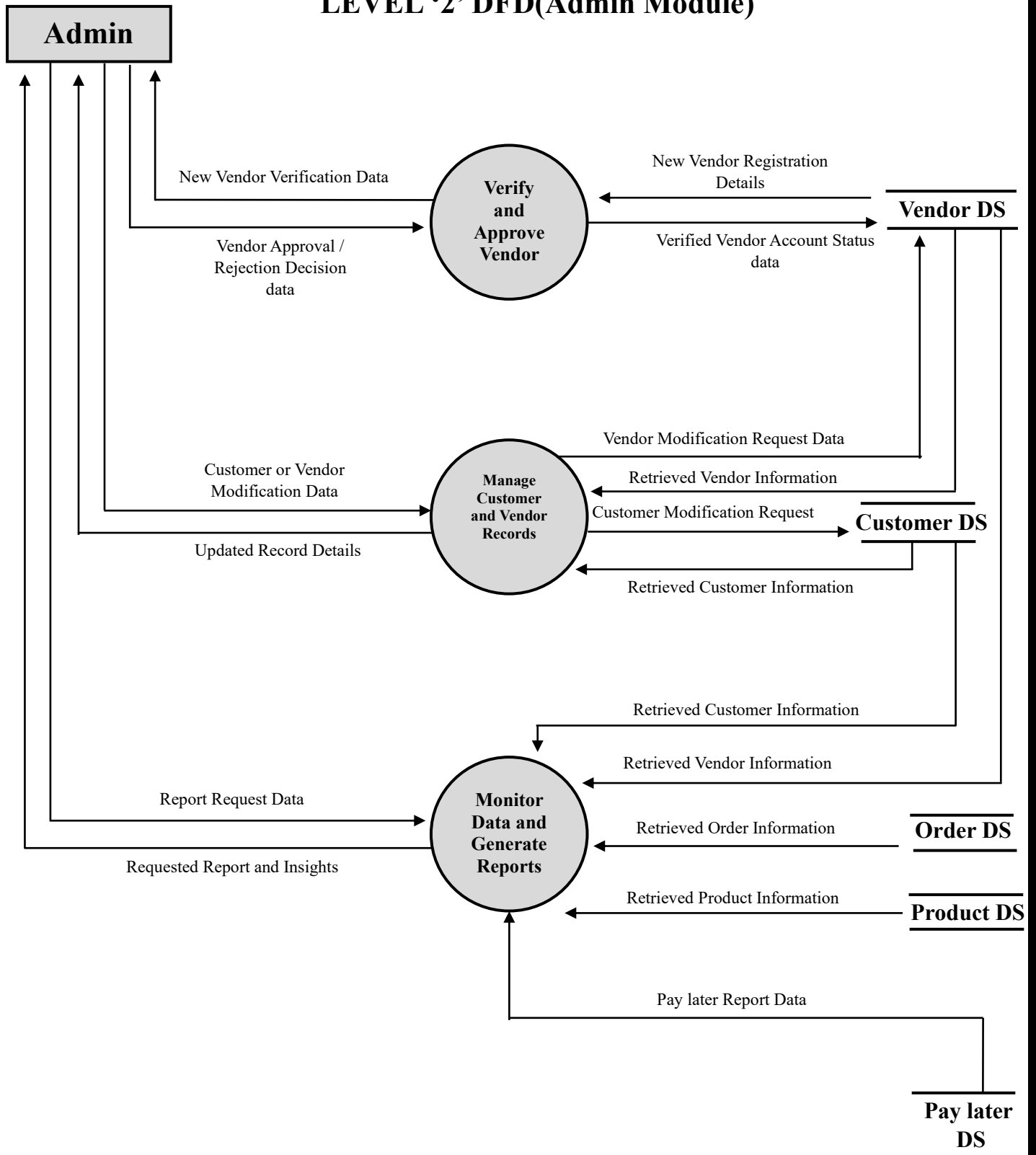
LEVEL '2' DFD (Vendor Module)



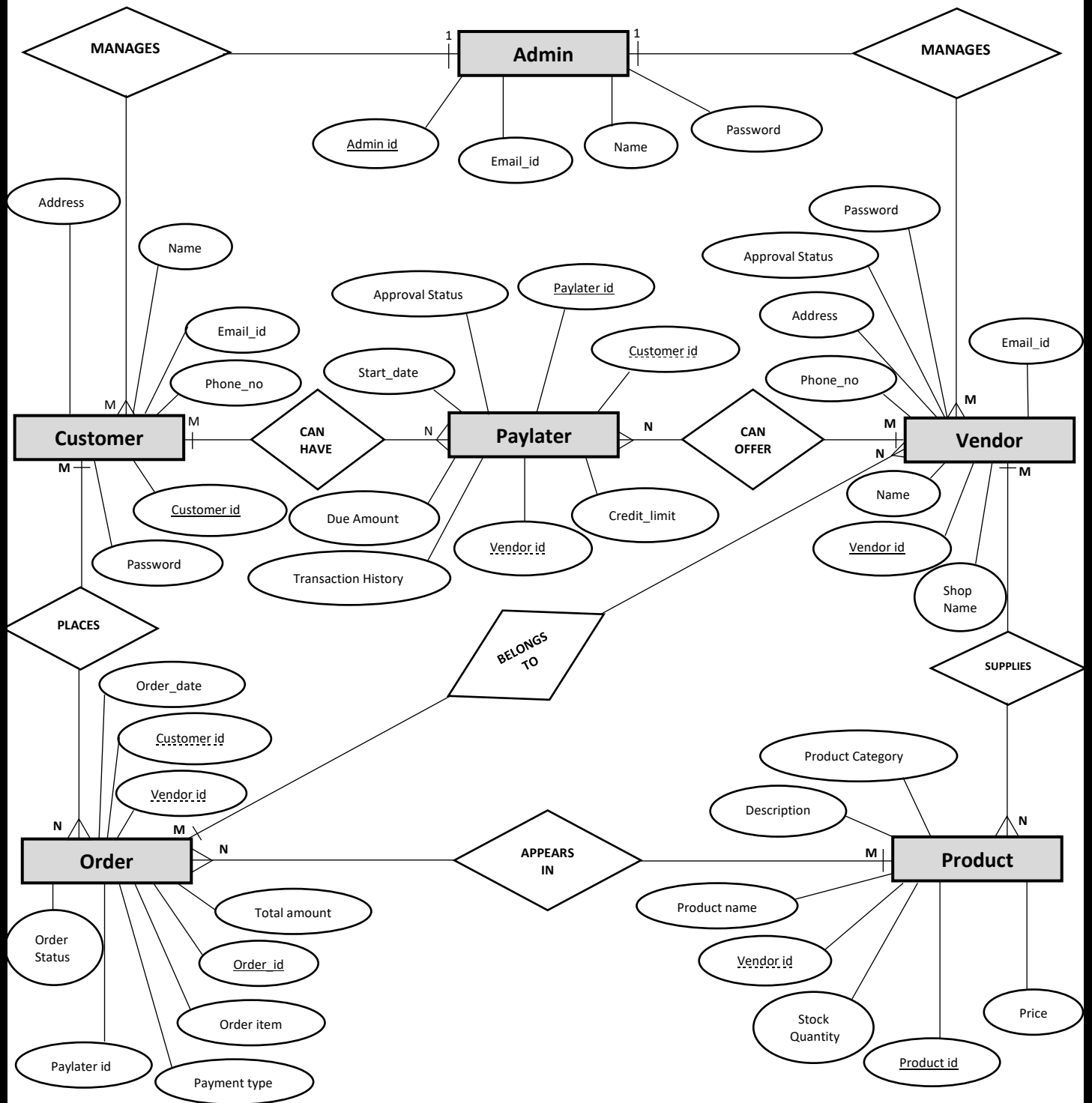
LEVEL '2' DFD (Customer Module)



LEVEL '2' DFD(Admin Module)



5.2 Entity Relationship (ER) Diagram



CODE SAMPLE

5.1 vendorsystem/urls.py

```
from django.contrib import admin
from django.urls import path,include
from django.conf import settings
from django.conf.urls.static import static

urlpatterns = [
    path('admin/', admin.site.urls),
    path("", include('accounts.urls')),
    path('products/', include('products.urls')),
    path('cart/', include('cart.urls')),
    path('order/', include('order.urls')),
    path('paylater/', include('paylater.urls')),
]

urlpatterns += static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)
```

5.2 accounts/models.py

```
from django.db import models
from django.core.validators import RegexValidator

phone_validator = RegexValidator(
    regex=r'^\d{10}$',
    message="The mobile number should be exactly 10 digits."
)

class Customer(models.Model):
    name = models.CharField(max_length=100)
    phone = models.CharField(
        max_length=10,
        unique=True,
        validators=[phone_validator]
    )
    email = models.EmailField()
    address = models.CharField(max_length=100)
```

```

password = models.CharField(max_length=100)
def __str__(self):
    return f'{self.name} ({self.phone})'
shop_name = models.CharField(max_length=100)
owner_name = models.CharField(max_length=100)
email = models.EmailField()
phone = models.CharField(
    max_length=10,
    unique=True,
    validators=[phone_validator]
)
locality = models.CharField(max_length=50)
password = models.CharField(max_length=255)
shop_image = models.ImageField(upload_to='shop_images/', null=True, blank=True)
license = models.FileField(upload_to='vendor_docs/')
identity_proof = models.FileField(upload_to='vendor_docs/')
is_approved = models.BooleanField(default=False)
created_at = models.DateTimeField(auto_now_add=True)
def __str__(self):
    return f'{self.shop_name} - {self.phone}'

```

5.3 accounts/views.py

```

from django.shortcuts import render, redirect, get_object_or_404
from django.contrib.auth.hashers import make_password, check_password
from django.db.models import Sum
from django.http import JsonResponse # AJAX ke liye zaroori hai
# Models Import
from .models import Customer, Vendor
from products.models import Product
from order.models import Order
from paylater.models import PayLaterRequest, PayLaterAccount

def home(request):

```

```

return render(request, "accounts/home.html")

def user_logout(request):
    request.session.flush()
    return redirect('accounts:home')

def cust_reg(request):
    if request.method == "POST":
        name = request.POST.get('name')
        phone = request.POST.get('phone')
        email = request.POST.get('email')
        address = request.POST.get('address')
        password = request.POST.get('password')
        confirm_password = request.POST.get('confirm_password')
        if not phone.isdigit() or len(phone) != 10:
            return render(request, 'customers/cust_reg.html', {'error': 'Phone number must be 10 digits.'})
        if Customer.objects.filter(phone=phone).exists():
            return render(request, 'customers/cust_reg.html', {'error': 'Phone already registered.'})
        if password != confirm_password:
            return render(request, 'customers/cust_reg.html', {'error': 'Passwords do not match.'})
        customer = Customer.objects.create(
            name=name, phone=phone, email=email,
            address=address, password=make_password(password)
        )
        request.session['customer_id'] = customer.id
        return redirect('accounts:customer_dashboard')
    return render(request, 'customers/cust_reg.html')

def cust_log(request):
    if request.method == "POST":
        phone = request.POST.get('phone')
        password = request.POST.get('password')
        try:
            user = Customer.objects.get(phone=phone)

```

```

    if check_password(password, user.password):
        request.session['customer_id'] = user.id
        return redirect('accounts:customer_dashboard')
    else:
        return render(request, "customers/cust_login.html", {'error': 'Invalid password'})
except Customer.DoesNotExist:
    return render(request, "customers/cust_login.html", {'error': 'User not found'})
return render(request, "customers/cust_login.html")

```

```
def ven_reg(request):
```

```

    if request.method == "POST":
        phone = request.POST.get('phone')
        password = request.POST.get('password')
        confirm_password = request.POST.get('confirm_password')
        if Vendor.objects.filter(phone=phone).exists():
            return render(request, 'vendors/ven_reg.html', {'error': 'Phone already registered.'})
        if password != confirm_password:
            return render(request, 'vendors/ven_reg.html', {'error': 'Passwords do not match'})
        Vendor.objects.create(
            shop_name=request.POST.get('shop_name'),
            owner_name=request.POST.get('owner_name'),
            email=request.POST.get('email'),
            phone=phone,
            locality=request.POST.get('locality'),
            shop_image=request.FILES.get('shop_image'),
            license=request.FILES.get('license'),
            identity_proof=request.FILES.get('identity'),
            password=make_password(password),
            is_approved=False
        )
        return render(request, 'vendors/ven_reg.html', {'success': True})
    return render(request, 'vendors/ven_reg.html')

```

```
def ven_log(request):
```

```

if request.method == "POST":
    shop_name = request.POST.get('shop_name')
    phone = request.POST.get('phone')
    password = request.POST.get('password')
    try:
        vendor = Vendor.objects.get(shop_name=shop_name, phone=phone)
        if not vendor.is_approved:
            return render(request, "vendors/ven_login.html", {'error': 'Account not approved yet'})
        if check_password(password, vendor.password):
            request.session['vendor_id'] = vendor.id
            return redirect('accounts:vendor_dashboard')
        else:
            return render(request, "vendors/ven_login.html", {'error': 'Invalid password'})
    except Vendor.DoesNotExist:
        return render(request, "vendors/ven_login.html", {'error': 'Vendor not found'})
return render(request, "vendors/ven_login.html")

```

5.4 accounts/base.html

```

{% load static %}
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>{% block title %}Local Vendor{% endblock %}</title>
    <link rel="stylesheet" href="{% static 'css/footer.css' %}">
    {% block extra_css %}{% endblock %}
</head>
<body>
    <div class="content-wrapper">
        {% block content %}
        {% endblock %}
    </div>

```

```

    {% include 'footer.html' %}
    {% block extra_js %} {% endblock %}
</body>
</html>

```

5.5 accounts/home.html

```

{% extends 'base.html' %}
{% load static %}
{% block title %}Home | Local Vendor Helping System{% endblock %}
{% block extra_css %}
<link rel="stylesheet" href="{% static 'css/accounts/style1 copy.css' %}">
{% endblock %}
{% block content %}
<nav class="navbar">
    <div class="container">
        <div class="navbar-container">
            <div class="logo-container">
                <a href="/" class="logo-link">
                    
                    <span class="logo-text">
                        <span class="loca">Loca</span><span class="cart">Cart</span>
                    </span>
                </a>
            </div>
            <div class="nav-links">
                <a href="#home">Home</a>
                <a href="#about">About</a>
                <a href="#features">Features</a>
                <a href="#contact">Contact</a>
                <div class="dropdown">
                    <button class="dropdown-btn" id="loginBtn">
                        Login
                        <svg width="16" height="16" viewBox="0 0 24 24">

```

```

        <polyline points="6 9 12 15 18 9"></polyline>
    </svg>
</button>
<div class="dropdown-menu" id="loginMenu">
    <a href="{% url 'accounts:cust_log' %}">Customer Login</a>
    <a href="{% url 'accounts:ven_log' %}">Vendor Login</a>
</div>
</div>
<div class="dropdown">
    <button class="dropdown-btn register" id="registerBtn">
        Register
        <svg width="16" height="16" viewBox="0 0 24 24">
            <polyline points="6 9 12 15 18 9"></polyline>
        </svg>
    </button>
    <div class="dropdown-menu" id="registerMenu">
        <a href="{% url 'accounts:cust_reg' %}">Customer Register</a>
        <a href="{% url 'accounts:ven_reg' %}">Vendor Register</a>
    </div>
</div>
</div>
<button class="mobile-menu-btn" id="mobileMenuBtn">
<svg id="menuIcon" width="24" height="24" viewBox="0 0 24 24" stroke="currentColor">
    <line x1="3" y1="12" x2="21" y2="12"></line>
    <line x1="3" y1="6" x2="21" y2="6"></line>
    <line x1="3" y1="18" x2="21" y2="18"></line>
</svg>
</button>
</div>
<div class="mobile-menu" id="mobileMenu">
    <a href="#home">Home</a>
    <a href="#about">About</a>
    <a href="#features">Features</a>
    <a href="#contact">Contact</a>

```

```

<div class="mobile-menu-section">
  <h4>Login</h4>
  <a href="{% url 'accounts:cust_log' %}">Customer Login</a>
  <a href="{% url 'accounts:ven_log' %}">Vendor Login</a>
</div>

<div class="mobile-menu-section">
  <h4>Register</h4>
  <a href="{% url 'accounts:cust_reg' %}">Customer Register</a>
  <a href="{% url 'accounts:ven_reg' %}">Vendor Register</a>
</div>
</div>
</nav>
<section id="home" class="hero">
  <div class="container">
    <div class="hero-grid">
      <div>
        <h1>Support Local Vendors.<br> Shop Smart.</h1>
        <p>Order from nearby shops, enjoy fast delivery, and use Pay Later.</p>
        <div class="location-picker">
          <select id="localitySelect" class="search-input-main">
            <option value="" disabled selected>Select Locality</option>
            <option value="Lanka">Lanka</option>
            <option value="Godowlia">Godowlia</option>
            <option value="Sarnath">Sarnath</option>
            <option value="Sigra">Sigra</option>
            <option value="Bhelupur">Bhelupur</option>
            <option value="Assi">Assi</option>
          </select>
          <div class="hero-buttons" style="margin-top: 15px;">
            <button onclick="fetchLocalData('vendor')" class="btn btn-primary">Show Vendors</button>
            <button onclick="fetchLocalData('product')" class="btn btn-secondary">Show Products</button>
          </div>
        </div>
      </div>
    </div>
  </div>

```

```

</div>
<div class="img">
    
</div>
</div>
</div>
</section>

```

5.6 paylater/models.py

```

import uuid
from django.db import models
from accounts.models import Customer, Vendor
class PayLaterRequest(models.Model):
    STATUS_CHOICES = [
        ('Pending', 'Pending'),
        ('Accepted', 'Accepted'),
        ('Declined', 'Declined'),
    ]
    customer = models.ForeignKey(Customer, on_delete=models.CASCADE)
    vendor = models.ForeignKey(Vendor, on_delete=models.CASCADE)
    status = models.CharField(max_length=20, choices=STATUS_CHOICES, default='Pending')
    request_date = models.DateTimeField(auto_now_add=True)
    def __str__(self):
        return f"Request: {self.customer.name} to {self.vendor.shop_name}"
class PayLaterAccount(models.Model):
    # Relational Fields
    customer = models.ForeignKey(Customer, on_delete=models.CASCADE)
    vendor = models.ForeignKey(Vendor, on_delete=models.CASCADE)
    account_id = models.CharField(max_length=20, unique=True, editable=False)
    credit_limit = models.FloatField(default=0.0)
    current_debt = models.FloatField(default=0.0) # Total amount to be paid
    is_active = models.BooleanField(default=True)

```

```

created_at = models.DateTimeField(auto_now_add=True)

def save(self, *args, **kwargs):
    if not self.account_id:
        self.account_id = "PL-" + str(uuid.uuid4().hex[:8].upper())
    super().save(*args, **kwargs)

@property
def remaining_limit(self):
    """Calculates available credit by subtracting debt from limit."""
    return self.credit_limit - self.current_debt

def __str__(self):
    return f'{self.account_id} | {self.customer.name} (@{self.vendor.shop_name})'

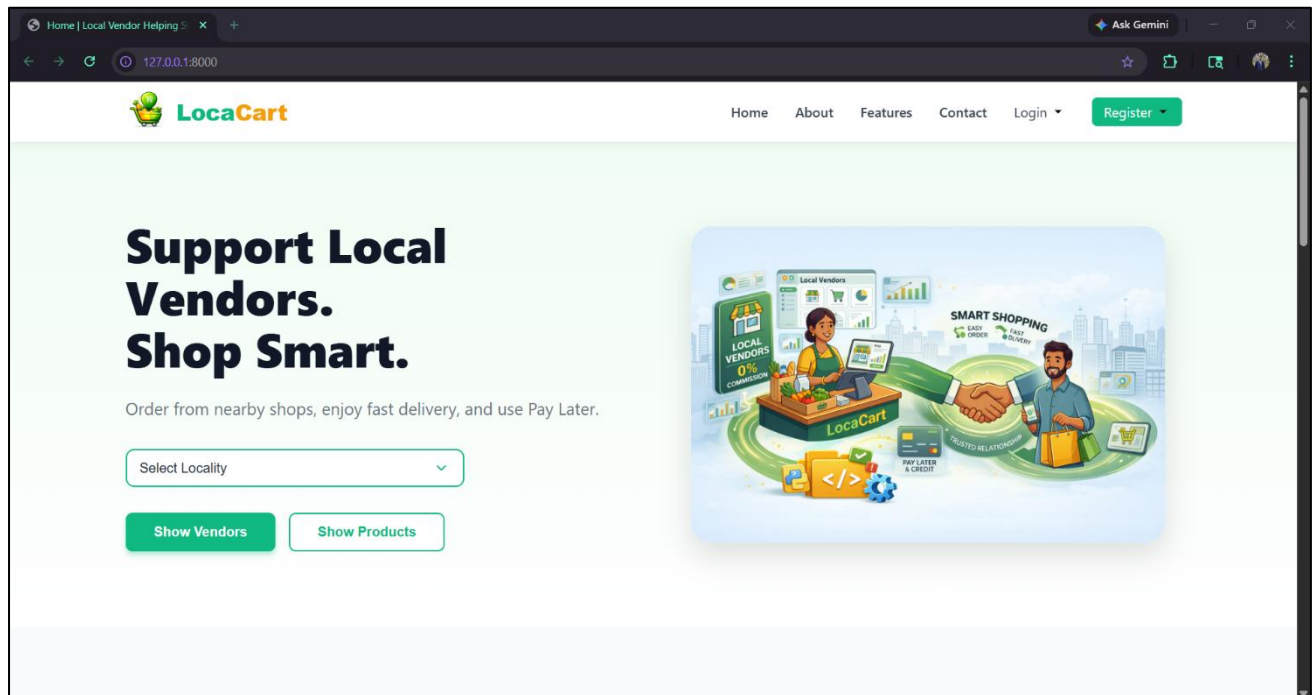
class PayLaterTransaction(models.Model):
    account = models.ForeignKey(
        PayLaterAccount,
        on_delete=models.CASCADE,
        related_name='transactions'
    )
    order_id = models.CharField(max_length=20) # External Order Reference
    amount = models.FloatField()
    description = models.TextField(blank=True, null=True)
    timestamp = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return f'TXN: {self.amount} for {self.account.account_id}'

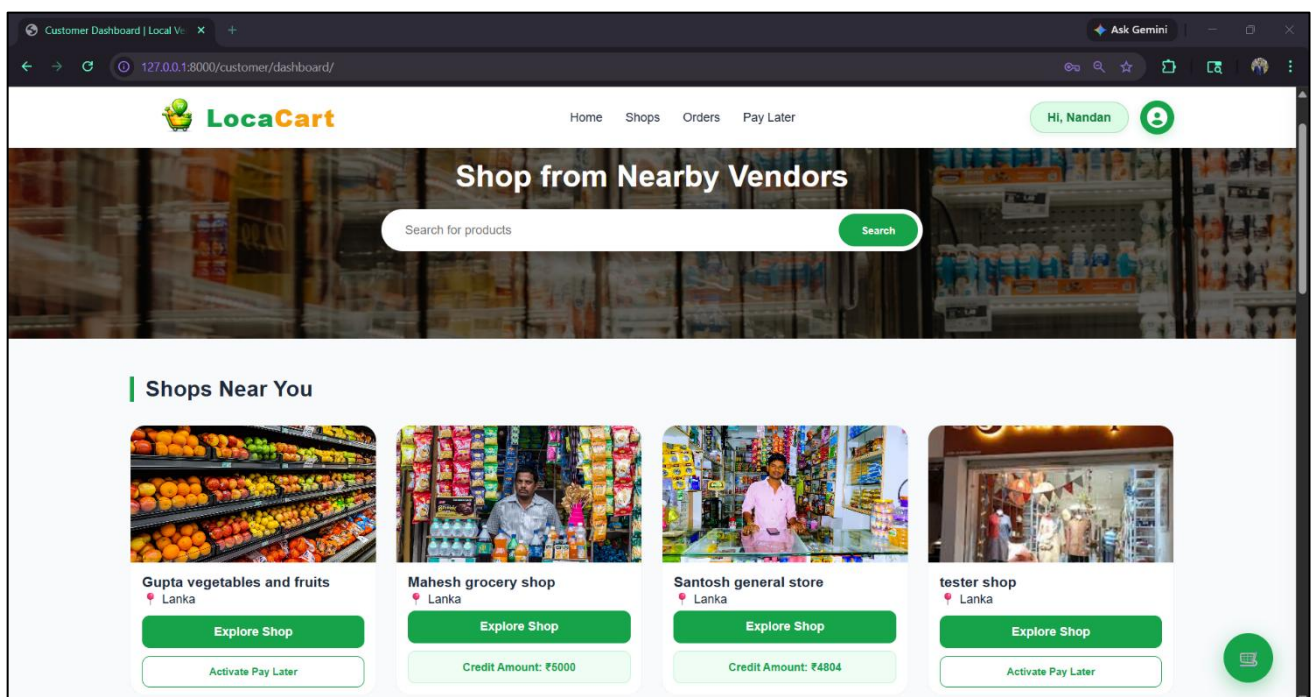
```

SOFTWARE SCREEN

6.1 Landing Page



6.2 Customer Dashboard



6.3 Vendor Registration Page

Vendor Registration
Register your shop to connect with local customers

SECTION A: BASIC DETAILS

Shop Name *
Enter shop name

Owner Name *
Enter owner name

Email Address *
Enter email address

Phone Number *
Enter registered phone number

Shop Locality *
Select Area

Upload Shop Image *
Choose File | No file chosen

SECTION B: SECURITY & VERIFICATION

Password *
Enter password

Confirm Password *
Confirm password

Shop License / GST *
Choose File | No file chosen

Identity Proof *
Choose File | No file chosen

Note: Your account will be activated only after admin approval.

REGISTER AS VENDOR

Already registered? [Login](#)

6.4 Vendor Dashboard

LocaCart Dashboard My Shop Orders

Store Overview

10 LIVE PRODUCTS

5 ACTIVE ORDERS

₹509.0 PAYMENT DUE

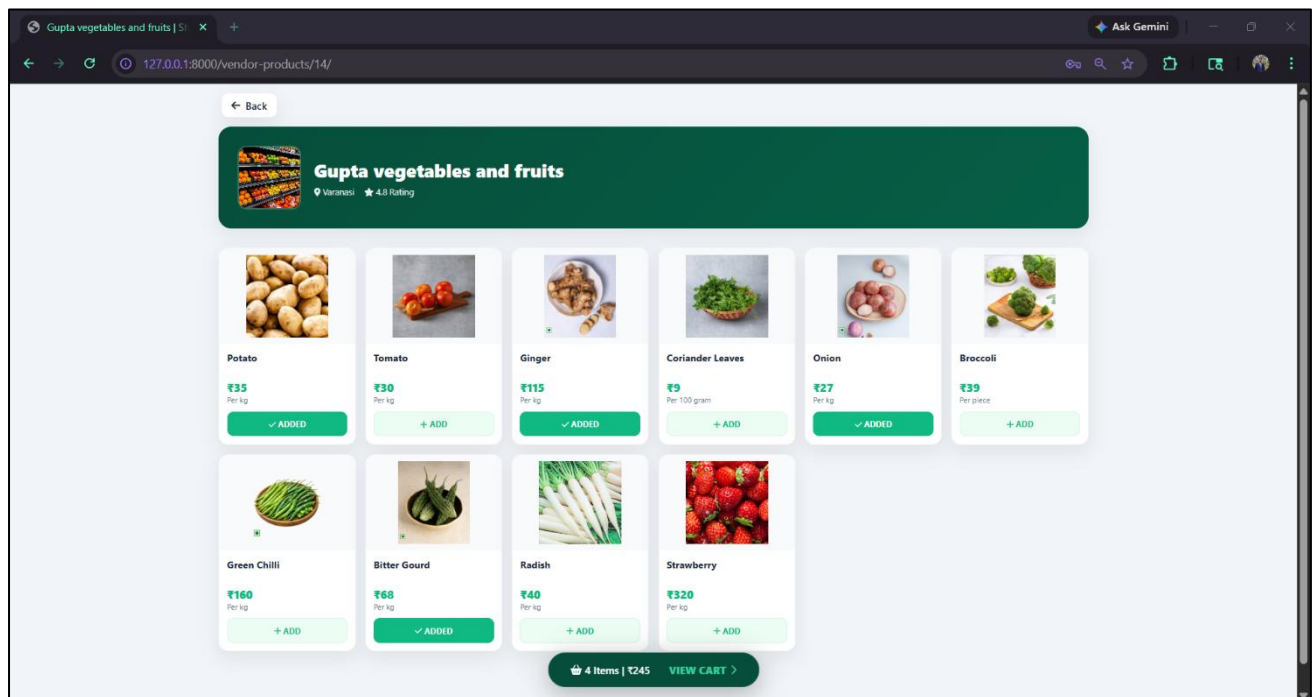
Common Tasks

Add Item Products Orders Pay Later

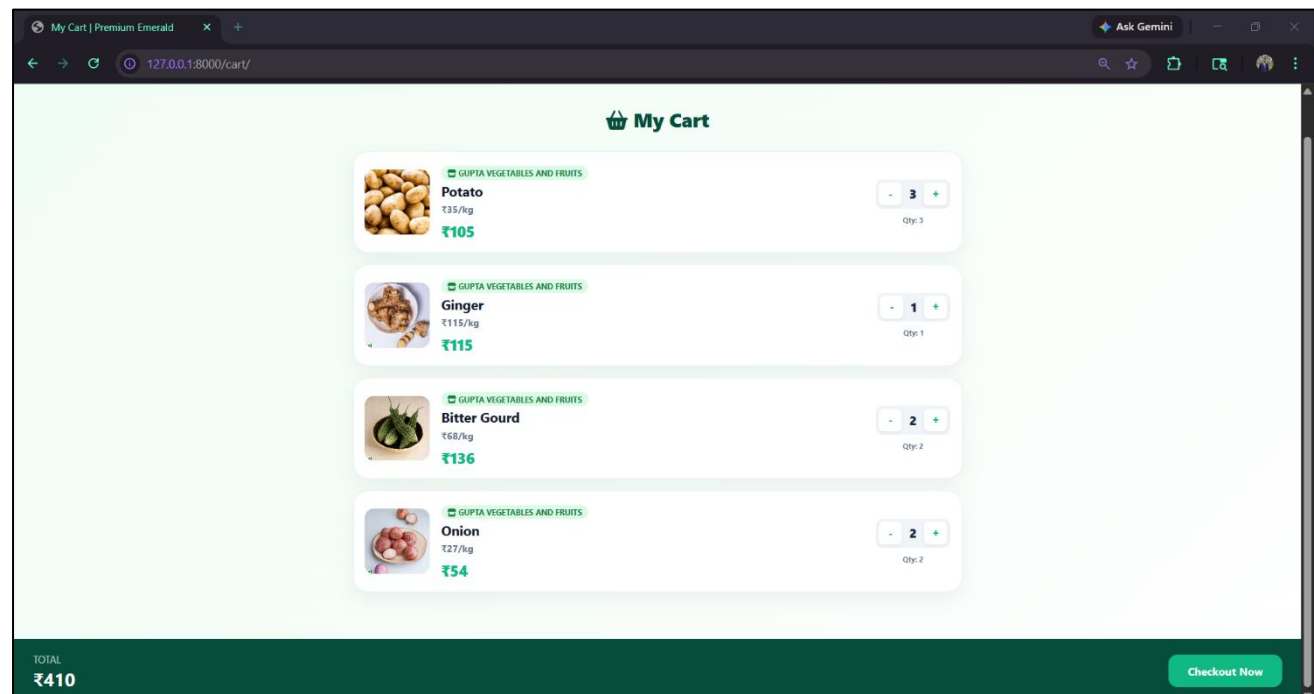
Products Preview Manage —

Product	Price	Weight
Strawberry	₹320	1 kg
Radish	₹40	1 kg
Bitter Gourd	₹68	1 kg
Green Chilli	₹160	1 kg
Broccoli	₹39	1 piece
Onion	₹27	1 kg

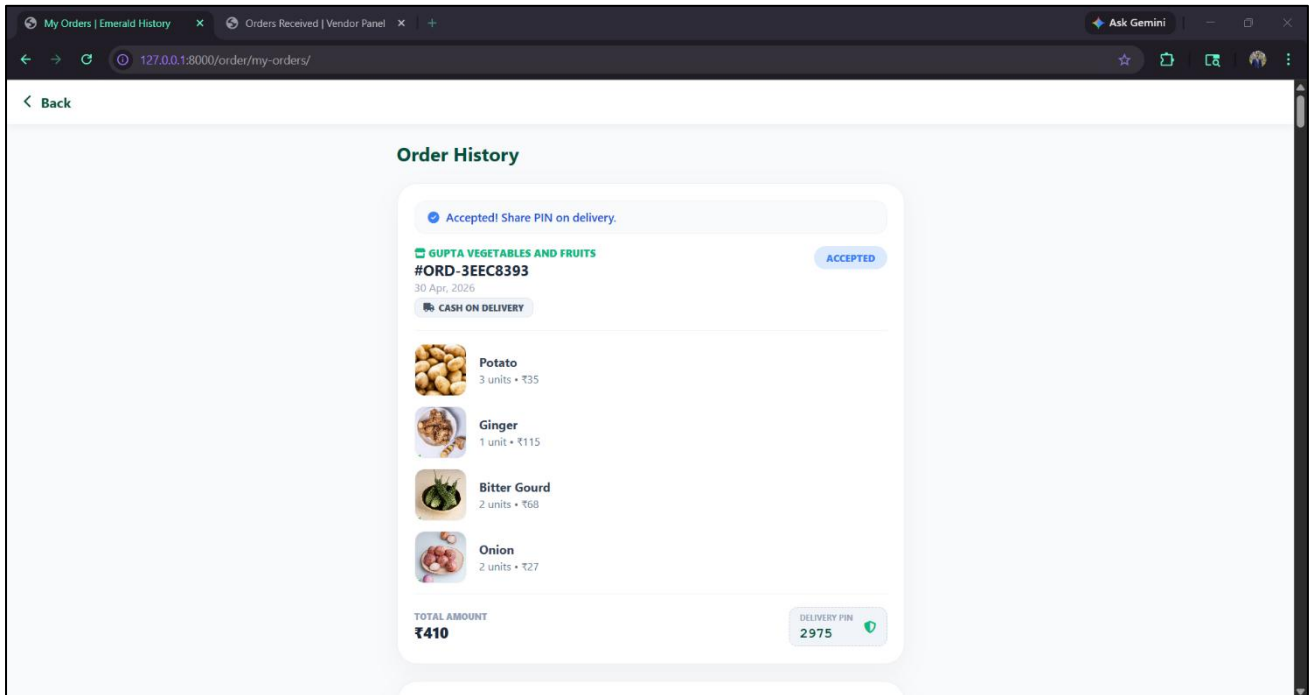
6.5 Customer Shopping Page



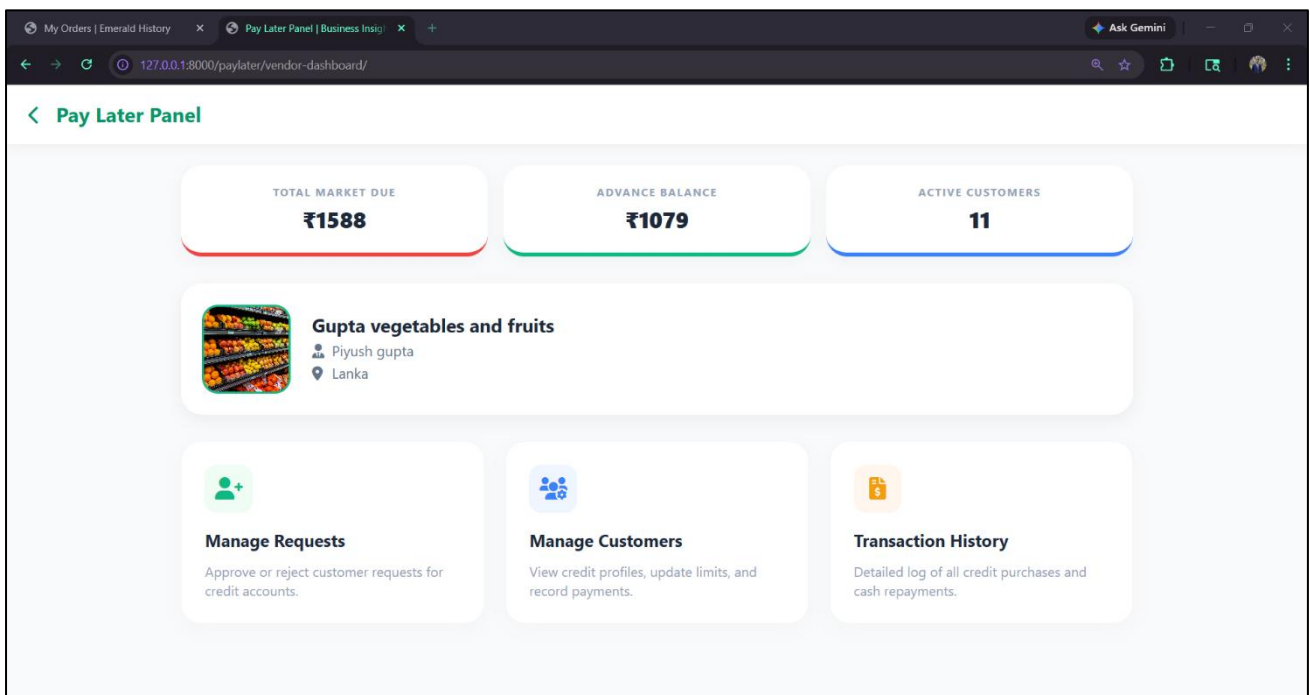
6.6 Customer Cart Page



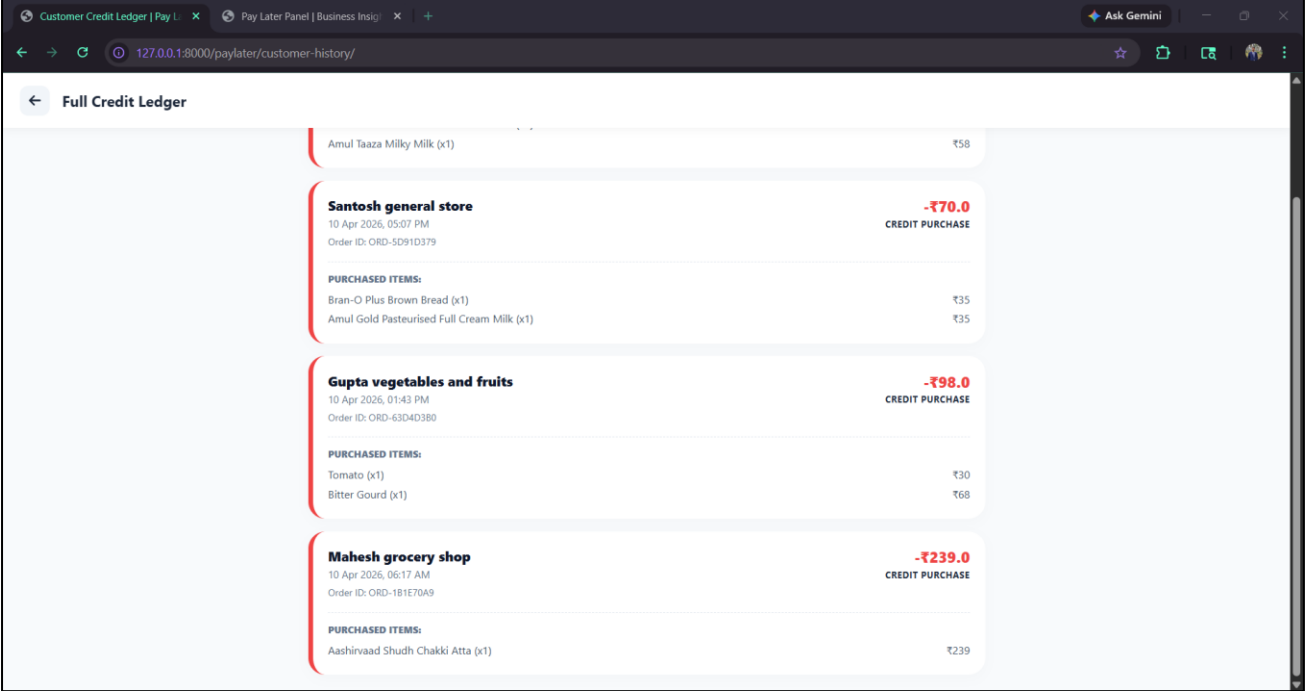
6.7 Customer Order History Page



6.8 Vendor Paylater Page

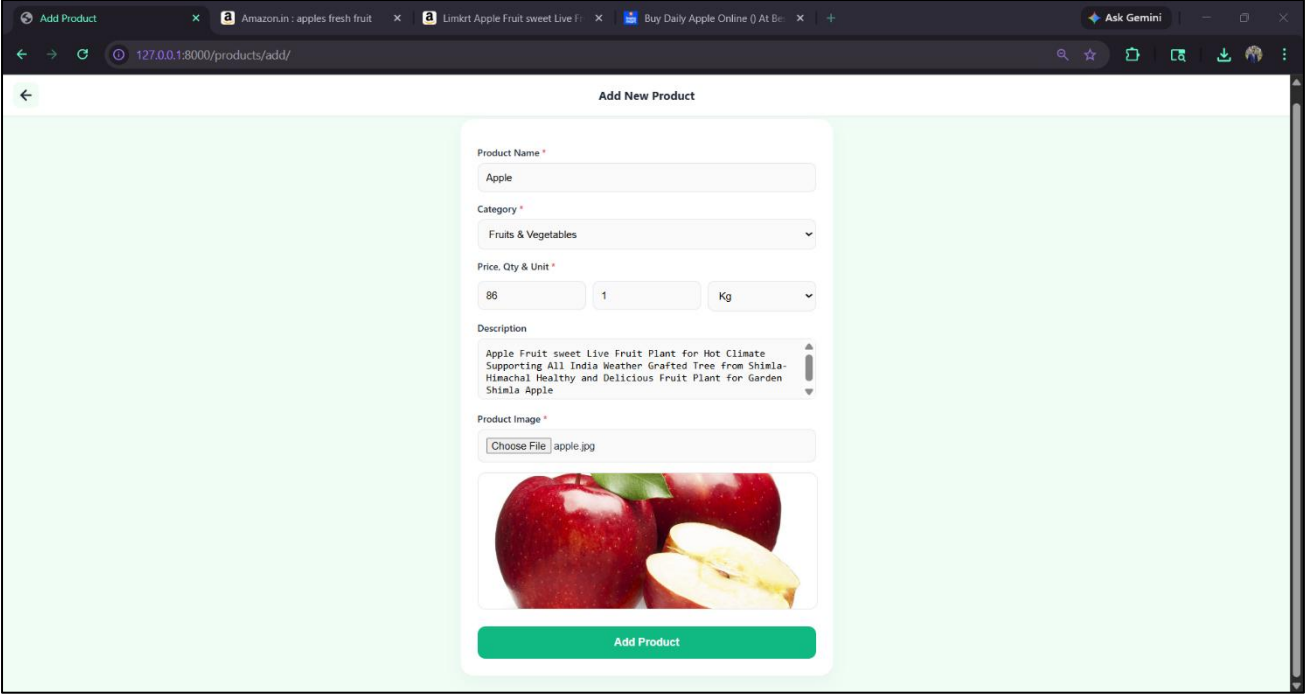


6.9 Customer Paylater History Page



Full Credit Ledger	
Amul Taaza Milky Milk (x1)	₹58
Santosh general store -₹70.0	
10 Apr 2026, 05:07 PM	
Order ID: ORD-5D91D379	
PURCHASED ITEMS:	
Bran-O Plus Brown Bread (x1)	₹35
Amul Gold Pasteurised Full Cream Milk (x1)	₹35
Gupta vegetables and fruits -₹98.0	
10 Apr 2026, 01:43 PM	
Order ID: ORD-63D4D380	
PURCHASED ITEMS:	
Tomato (x1)	₹30
Bitter Gourd (x1)	₹68
Mahesh grocery shop -₹239.0	
10 Apr 2026, 06:17 AM	
Order ID: ORD-1B1E70A9	
PURCHASED ITEMS:	
Aashirvaad Shudh Chakki Atta (x1)	₹239

6.10 Vendor Add New Product Page



Add New Product

Product Name *
Apple

Category *
Fruits & Vegetables

Price, Qty & Unit *
86 1 Kg

Description
Apple Fruit sweet Live Fruit Plant for Hot Climate
Supporting All India Weather Grafted Tree from Shimla-
Himachal Healthy and Delicious Fruit Plant for Garden
Shimla Apple

Product Image *
Choose File | apple.jpg



Add Product

CONCLUSION

The Local Vendor Helping System successfully addresses the growing challenges faced by local shopkeepers in the era of rapidly expanding online delivery platforms. The project demonstrates how technology can be used not to replace traditional neighbourhood businesses, but to empower and modernize them while preserving trust-based relationships between vendors and customers.

Through this system, local vendors are provided with a digital platform to manage product catalogs, process customer orders, and maintain transparent records of sales and payments. Customers benefit from the convenience of online ordering, minimal or no delivery charges, easy returns, and the flexibility of a Pay Later facility that digitizes the traditional udhari system. This combination of modern digital tools with familiar market practices ensures higher acceptance among users.

The inclusion of a secure Pay Later mechanism is a significant achievement of the project, as it brings transparency and accountability to credit-based transactions. Vendors retain full control over customer approvals and credit limits, reducing financial risk while improving customer loyalty. The Admin module further ensures system reliability by verifying vendors, managing accounts, and generating analytical reports.

From a technical and operational perspective, the system is feasible, cost-effective, and scalable. The use of open-source technologies such as Django, HTML, CSS, JavaScript, and SQLite makes the system easy to implement and maintain. The modular architecture allows for future enhancements such as mobile application integration, advanced analytics, and AI-based demand prediction.

In conclusion, the Local Vendor Helping System provides a sustainable, practical, and socially beneficial solution that strengthens local economies, supports small businesses, and delivers convenience to customers. The project effectively fulfils its objectives and serves as a strong foundation for future expansion and real-world deployment.

FUTURE SCOPE

The Local Vendor Helping System has strong potential for enhancement and large-scale adoption. Although the current system effectively meets its objectives, several future improvements can further increase its usability, scalability, and impact.

One of the major future enhancements is the development of a dedicated mobile application for Android and iOS platforms. A mobile app would improve accessibility for both vendors and customers, especially for users who primarily rely on smartphones. Push notifications for order updates, Pay Later reminders, and delivery status can significantly enhance user engagement.

The system can also be expanded by integrating advanced delivery management features, such as delivery boy assignment, live tracking, and optimized delivery routes. This would further improve delivery efficiency and customer satisfaction, particularly in densely populated localities.

Another important future scope is the implementation of AI and data analytics. Sales data and customer behaviour can be analysed to provide vendors with insights such as demand forecasting, best-selling products, inventory recommendations, and personalized offers. AI-based credit risk analysis can also be introduced to assist vendors in making informed Pay Later approval decisions.

The Pay Later module can be enhanced by integrating online payment gateways, automated reminders for due payments, and digital invoices. This would further improve financial transparency and reduce manual follow-ups by vendors.

In addition, the system can be scaled to support multiple cities and regions, enabling broader adoption. Migration from SQLite to enterprise-level databases such as MySQL or PostgreSQL and deployment on cloud platforms can support high traffic and large datasets.

Finally, the platform can be extended to include other types of local vendors, such as pharmacies, stationery shops, local food vendors, and service providers, making it a comprehensive hyper-local commerce ecosystem.

In summary, the future scope of the Local Vendor Helping System is vast. With continuous enhancement and technological integration, the system can evolve into a powerful digital solution that strengthens local businesses, promotes self-reliance, and contributes to sustainable economic growth.

BIBLIOGRAPHY

1. Aggarwal, K. K. (2005). *Software Engineering*. New Age International Publishers, New Delhi.
2. Gupta, A. K. (2007). *Taxonomy of Database Management Systems*. Firewall Media, New Delhi.
3. Sommerville, I. (2016). *Software Engineering* (10th ed.). Pearson Education.
4. Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). *Database System Concepts* (7th ed.). McGraw-Hill Education.
5. GeeksforGeeks. (n.d.). *What is a Data Flow Diagram (DFD)?* Retrieved from <https://www.geeksforgeeks.org/software-engineering/what-is-dfddata-flow-diagram/>
6. GeeksforGeeks. (n.d.). *Introduction of ER Model*. Retrieved from <https://www.geeksforgeeks.org/dbms/introduction-of-er-model/>
7. Django Software Foundation. (n.d.). *Django Documentation*. Retrieved from <https://docs.djangoproject.com/>
8. W3Schools. (n.d.). *HTML, CSS and JavaScript Tutorials*. Retrieved from <https://www.w3schools.com/>
9. Pressman, R. S., & Maxim, B. R. (2020). *Software Engineering: A Practitioner's Approach* (9th ed.). McGraw-Hill Education.
10. YouTube – Gate Smashers. (n.d.). *Software Engineering and DBMS Tutorials*.